



Обзор RocksDB

[Прыжок на дно](#)Itamasi отредактировал эту страницу 18 июля 2023 года · [10 правок](#)

🔗 1. Введение

RocksDB появился в [Facebook](#) как механизм хранения данных для серверных рабочих нагрузок на различных носителях, изначально ориентированный на быстрое хранение (особенно на флеш-накопителях). Это библиотека на C++ для хранения ключей и значений, которые представляют собой байтовые потоки произвольного размера. Она поддерживает как точечный поиск, так и сканирование по диапазону, а также различные типы гарантий ACID.

Соблюдается баланс между возможностью настройки и самоадаптируемостью. RocksDB обладает очень гибкими настройками конфигурации, которые можно адаптировать для работы в различных производственных средах, включая твердотельные накопители, жесткие диски, ramfs или удаленные хранилища. Она поддерживает различные алгоритмы сжатия и предоставляет хорошие инструменты для поддержки и отладки в производственной среде. С другой стороны, мы также стараемся ограничить количество параметров, обеспечить достаточно высокую производительность «из коробки» и использовать адаптивные алгоритмы там, где это возможно.

RocksDB заимствует значительный объем кода из проекта с открытым исходным кодом [leveldb](#), а также идеи из [Apache HBase](#). Первоначальный код был создан на основе open source leveldb 1.5. Он также основан на коде и идеях, разработанных в Facebook до появления RocksDB.

🔗 2. Предположения и цели

🔗 Производительность:

Основная идея RocksDB заключается в том, что она должна быть производительной для быстрого хранения данных и серверных рабочих нагрузок. Она должна поддерживать эффективный точечный поиск, а также сканирование по диапазону. Она должна быть настраиваемой, чтобы поддерживать рабочие нагрузки с большим количеством случайных чтений, рабочие нагрузки с большим количеством обновлений или их комбинацию. Ее архитектура должна позволять легко находить компромиссы для различных рабочих нагрузок и аппаратного обеспечения.

🔗 Поддержка производства:

RocksDB должна быть спроектирована таким образом, чтобы иметь встроенную поддержку инструментов и утилит, облегчающих развертывание и отладку в производственных средах. Если система хранения данных пока не может автоматически адаптироваться к приложению и оборудованию, мы предоставим пользователям некоторые параметры для настройки производительности.

🔗 Совместимость:

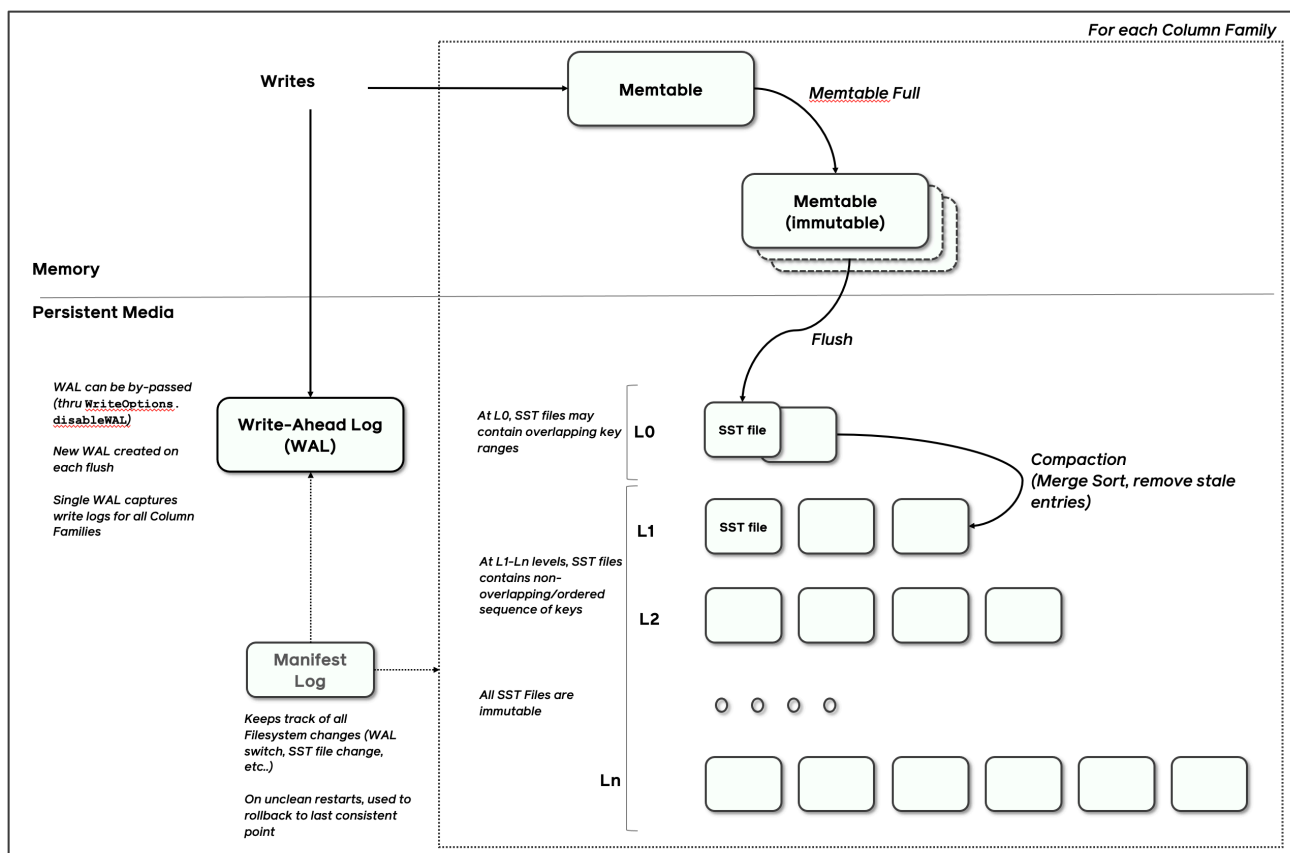
Новые версии этого программного обеспечения должны быть обратно совместимы, чтобы при обновлении до новых версий RocksDB не требовалось вносить изменения в существующие приложения. Если в существующих приложениях не используются новые функции, они также должны иметь возможность вернуться к последней старой версии. См. [Совместимость RocksDB с разными версиями](#).

🔗 3. Архитектура высокого уровня

RocksDB — это библиотека для хранения данных с интерфейсом «ключ-значение», где ключи и значения представляют собой произвольные байтовые потоки. RocksDB организует все данные в отсортированном порядке, а основными операциями являются `Get(key)`, `NewIterator()`, `Put(key, val)`, `Delete(key)`, и `SingleDelete(key)`.

Три основные конструкции RocksDB — это **memtable**, **sstfile** и **logfile**. [Memtable](#) — это структура данных в оперативной памяти. Новые записи вставляются в *memtable* и при необходимости записываются в [logfile \(он же. журнал предзаписи \(WAL\)\)](#). Logfile — это файл в хранилище, записи в который записываются последовательно. Когда *memtable* заполняется, она сбрасывается в [sstfile](#) в хранилище, а соответствующий logfile можно безопасно удалить. Данные в *sstfile* отсортированы для удобства поиска ключей.

Формат *sstfile* по умолчанию описан более подробно [здесь](#).



4. Особенности

Семейства столбцов

RocksDB поддерживает разделение экземпляра базы данных на несколько семейств столбцов. Все базы данных создаются с семейством столбцов по умолчанию, которое используется для операций, в которых семейство столбцов не указано.

RocksDB гарантирует пользователям согласованность данных в семействах столбцов, в том числе после аварийного восстановления, если включена функция WAL или атомарная очистка. Она также поддерживает атомарные операции между семействами столбцов через `WriteBatch` API.

Обновления

А `Put` API вставляет в базу данных одно значение ключа. Если ключ уже существует в базе данных, предыдущее значение будет перезаписано. А `Write` API позволяет атомарно вставлять, обновлять или удалять в базе данных несколько значений ключей. База данных гарантирует, что либо все значения ключей, указанные в одном вызове `Write` API, будут вставлены в базу данных, либо ни одно из них не будет вставлено. Если какой-либо из этих ключей уже существует в базе данных, предыдущие значения будут перезаписаны. [DeleteRange](#) API можно использовать для удаления всех ключей из диапазона.

Получение значений, итераторы и моментальные снимки

☞ Ключи и значения рассматриваются как чистые байтовые потоки. Размер ключа или значения не ограничен. API `Get` позволяет приложению получить из базы данных одно значение ключа. API `MultiGet` позволяет приложению получить из базы данных несколько значений ключей. Все значения ключей, возвращаемые при вызове `MultiGet`, согласованы между собой.

Все данные в базе логически упорядочены. Приложение может указать метод сравнения ключей, который определяет общий порядок ключей. API `Iterator` позволяет приложению выполнять сканирование диапазона в базе данных.

`Iterator` может выполнить поиск по указанному ключу, после чего приложение может начать сканирование по одному ключу за раз, начиная с этой точки. API `Iterator` также можно использовать для обратного перебора ключей в базе данных. При создании `Iterator` создается согласованное представление базы данных на определенный момент времени. Таким образом, все ключи, возвращаемые через `Iterator`, относятся к согласованному представлению базы данных.

[Snapshot](#) API позволяет приложению создать представление базы данных на определенный момент времени. Для чтения данных из указанного снимка можно использовать `Get` и `Iterator` API. В некотором смысле `Snapshot` и `Iterator` обеспечивают представление базы данных на определенный момент времени, но их реализация отличается. Кратковременные сканирования в режиме переднего плана лучше выполнять с помощью итератора, а длительные сканирования в фоновом режиме — с помощью снимка. `Iterator` поддерживает счетчик ссылок на все базовые файлы, соответствующие текущему состоянию базы данных. Эти файлы не удаляются до тех пор, пока `Iterator` не будет освобожден. С другой стороны, `Snapshot` не предотвращает удаление файлов. Вместо этого процесс сжатия учитывает наличие `Snapshots` и обещает никогда не удалять ключ, который виден в любом существующем `Snapshot`.

`Snapshots` are not persisted across database restarts: a reload of the RocksDB library (via a server restart) releases all pre-existing `Snapshots`.

☞ Transactions

RocksDB supports multi-operational transactions. It supports both of optimistic and pessimistic mode. See [Transactions](#).

☞ Prefix Iterators

Most LSM-tree engines cannot support an efficient range scan API because it needs to look into multiple data files. But, most applications do not do pure-random scans of key ranges in the database; instead, applications typically scan within a key-prefix. RocksDB uses this to its advantage. Applications can configure a `Options.prefix_extractor` to enable a key-prefix based filtering. When `Options.prefix_extractor` is set, a hash of the prefix is also added to the Bloom. An `Iterator` that specifies a key-prefix (in `ReadOptions`) will use the [Bloom Filter](#) to avoid looking into data files that do not contain keys with the specified key-prefix. See [Prefix-Seek](#).

🔗 Persistence

RocksDB has a [Write Ahead Log \(WAL\)](#). All write operations (`Put` , `Delete` and `Merge`) are stored in an in-memory buffer called the memtable as well as optionally inserted into WAL. On restart, it re-processes all the transactions that were recorded in the log.

WAL can be configured to be stored in a directory different from the directory where the SST files are stored. This is necessary for those cases in which you might want to store all data files in non-persistent fast storage. At the same time, you can ensure no data loss by putting all transaction logs on slower but persistent storage.

Each `Put` has a flag, set via `WriteOptions` , which specifies whether or not the `Put` should be inserted into the transaction log. The `WriteOptions` may also specify whether or not a `fsync` call is issued to the transaction log before a `Put` is declared to be committed.

Internally, RocksDB uses a batch-commit mechanism to batch transactions into the log so that it can potentially commit multiple transactions using a single `fsync` call.

🔗 Data Checksumming

RocksDB uses a checksum to detect corruptions in storage. These checksums are for each SST file block (typically between 4K to 128K in size). A block, once written to storage, is never modified. RocksDB also maintains a Full File Checksum (see [Full File Checksum and Checksum Handoff](#)) and optionally [per key-value checksum](#).

RocksDB dynamically detects and utilizes CPU checksum offload support.

🔗 Multi-Threaded Compactions

In the presence of ongoing writes, compactions are needed for space efficiency, read (query) efficiency, and timely data deletion. Compaction removes key-value bindings that have been deleted or overwritten, and re-organizes data for query efficiency. Compactions may occur in multiple threads if configured.

The entire database is stored in a set of *sstfiles*. When a *memtable* is full, its content is written out to a file in Level-0 (L0) of the LSM tree. RocksDB removes duplicate and overwritten keys in the memtable when it is flushed to a file in L0. In *compaction*, some files are periodically read in and merged to form larger files, often going into the next LSM level (such as L1, up to Lmax).

The overall write throughput of an LSM database directly depends on the speed at which compactions can occur, especially when the data is stored in fast storage like SSD or RAM. RocksDB may be configured to issue concurrent compaction requests from multiple threads. It is observed that sustained write rates may increase by as much as a factor of 10 with multi-threaded compaction when the database is on SSDs, as compared to single-threaded compactions.

[Compaction Styles](#)

Both Level Style Compaction and Universal Style Compaction store data in a fixed number of logical *levels* in the database. More recent data is stored in Level-0 (L0) and older data in higher-numbered levels, up to Lmax. Files in L0 may have overlapping keys, but files in other levels generally form a single sorted run per level.

Level Style Compaction (default) typically optimizes disk footprint vs. logical database size (space amplification) by minimizing the files involved in each compaction step: merging one file in L_n with all its overlapping files in L_{n+1} and replacing them with new files in L_{n+1} .

Universal Style Compaction typically optimizes total bytes written to disk vs. logical database size (write amplification) by merging potentially many files and levels at once, requiring more temporary space. Universal typically results in lower write-amplification but higher space- and read-amplification than Level Style Compaction.

FIFO Style Compaction drops oldest file when obsolete and can be used for cache-like data. In FIFO compaction, all files are in level 0. When total size of the data exceeds configured size (`CompactionOptionsFIFO::max_table_files_size`), we delete the oldest table file.

We also enable developers to develop and experiment with custom compaction policies. For this reason, RocksDB has appropriate hooks to switch off the inbuilt compaction algorithm and has other APIs to allow applications to operate their own compaction algorithms.

`Options.disable_auto_compaction`, if set, disables the native compaction algorithm. The `GetLiveFilesMetaData` API allows an external component to look at every data file in the database and decide which data files to merge and compact. Call `CompactFiles` to compact files you want. The `DeleteFile` API allows applications to delete data files that are deemed obsolete.

Metadata storage

A manifest log file is used to record all the database state changes. The compaction process adds new files and deletes existing files from the database, and it makes these operations persistent by recording them in the [MANIFEST](#).

🔗 Avoiding Stalls

Background compaction threads are also used to flush *memtable* contents to a file on storage. If all background compaction threads are busy doing long-running compactions, then a sudden burst of writes can fill up the *memtable(s)* quickly, thus stalling new writes. This situation can be avoided by configuring RocksDB to keep a small set of threads explicitly reserved for the sole purpose of flushing *memtable* to storage.

🔗 Compaction Filter

Some applications may want to process keys at compaction time. For example, a database with inherent support for time-to-live (TTL) may remove expired keys. This can be done via an application-defined [Compaction-Filter](#). If the application wants to continuously delete data older than a specific time, it can use the compaction filter to drop records that have expired. The RocksDB Compaction Filter gives control to the application to modify the value of a key or to drop a key entirely as part of the compaction process. For example, an application can continuously run a data sanitizer as part of the compaction.

🔗 ReadOnly Mode

A database may be opened in ReadOnly mode, in which the database guarantees that the application may not modify anything in the database. This results in much higher read performance because oft-traversed code paths avoid locks completely.

🔗 Database Debug Logs(aka. Info Logs)

By default, RocksDB writes detailed logs to a file named LOG*. These are mostly used for debugging and analyzing a running system. Users can choose different log levels (see `DBOptions.info_log_level`). This LOG may be configured to roll at a specified periodicity. The logging interface is pluggable. Users can plug in a different logger. See [Logger](#).

🔗 Data Compression

RocksDB supports lz4, zstd, snappy, zlib, and lz4_hc compression, as well as xpress under Windows. RocksDB may be configured to support different compression algorithms for data at the bottommost level, where 90% of data lives. A typical installation might configure ZSTD (or Zlib if not available) for the bottom-most level and LZ4 (or Snappy if it is not available) for other levels. See [Compression](#).

🔗 Full Backups and Replication

RocksDB provides a backup API, `BackupEngine`. You can read more about it here: [How to backup RocksDB](#)

RocksDB itself is not a replicated, but it provides some helper functions to enable users to implement their replication system on top of RocksDB, see [Replication Helpers](#).

🔗 Support for Multiple Embedded Databases in the same process

A common use-case for RocksDB is that applications inherently partition their data set into logical partitions or shards. This technique benefits application load balancing and fast recovery from faults. This means that a single server process should be able to operate multiple RocksDB databases simultaneously. This is done via an environment object named `Env`. Among other things, a thread pool is associated with an `Env`. If applications want to share a common thread pool (for background compactions) among multiple database instances, then it should use the same `Env` object for opening those databases.

Similarly, multiple database instances may share the same block cache or rate limiter.

🔗 Block Cache -- Compressed and Uncompressed Data

RocksDB uses a [LRU cache for blocks](#) to serve reads. The block cache is partitioned into two individual caches: the first caches uncompressed blocks and the second caches compressed blocks in RAM. If a compressed block cache is configured, users may wish to enable direct I/O to prevent redundant caching of the same data in OS page cache.

🔗 Table Cache

The Table Cache is a construct that caches open file descriptors. These file descriptors are for **sstfiles**. An application can specify the maximum size of the Table Cache, or configure RocksDB to always keep all files open, to achieve better performance.

🔗 I/O Control

RocksDB allows users to configure I/O from and to SST files in different ways. Users can enable direct I/O so that RocksDB takes full control to the I/O and caching. An alternative is to leverage some options to allow users to hint about how I/O should be executed. They can suggest RocksDB to call `fadvise` in files to read, call periodic range sync in files being appended, enable direct I/O, etc... See [IO](#) for more details.

🔗 Stackable DB

RocksDB has a built-in wrapper mechanism to add functionality as a layer above the code database kernel. This functionality is encapsulated by the `StackableDB` API. For example, the time-to-live functionality is implemented by a `StackableDB` and is not part of the core RocksDB API. This approach keeps the code modularized and clean.

🔗 Memtables:

🔗 Pluggable Memtables:

The default implementation of the memtable for RocksDB is a skiplist. The skiplist is a sorted set, which is a necessary construct when the workload interleaves writes with range-scans. Some applications do not interleave writes and scans, however, and some applications do not do range-scans at all. For these applications, a sorted set may not provide optimal performance. For this reason, RocksDB's memtable is pluggable. Some alternative implementations are provided. Three memtables are part of the library: a skiplist memtable, a vector memtable and a prefix-hash memtable. A vector memtable is appropriate for bulk-loading data into the database. Every write inserts a new element at the end of the vector; when it is time to flush the memtable to storage the elements in the vector are sorted and written out to a file in L0. A prefix-hash memtable allows efficient processing of gets, puts and scans-within-a-key-prefix. Although the pluggability of memtable is not provided as a public API, it is possible for an application to provide its own implementation of a memtable, in a private fork.

🔗 Memtable Pipelining

RocksDB supports configuring an arbitrary number of memtables for a database. When a memtable is full, it becomes an immutable memtable and a background thread starts flushing its contents to storage. Meanwhile, new writes continue to accumulate to a newly allocated memtable. If the newly allocated memtable is filled up to its limit, it is also converted to an immutable memtable and is inserted into the flush pipeline. The background thread continues to flush all the pipelined immutable memtables to storage. This pipelining increases write throughput of RocksDB, especially when it is operating on slow storage devices.

🔗 Garbage Collection during Memtable Flush:

When a memtable is being flushed to storage, an inline-compaction process is executed. Garbages are removed in the same way as compactions. Duplicate updates for the same key are removed from the output stream. Similarly, if an earlier put is hidden by a later delete, then the put is not written to the output file at all. This feature reduces the size of data on storage and write amplification greatly, for some workloads.

🔗 Merge Operator

RocksDB natively supports three types of records, a `Put` record, a `Delete` record and a `Merge` record. When a compaction process encounters a `Merge` record, it invokes an application-specified method called the Merge Operator. The Merge can combine multiple `Put` and `Merge` records into a single one. This powerful feature allows applications that typically do read-modify-writes to avoid the reads altogether. It allows an application to record the intent-of-the-operation as a `Merge Record`, and the RocksDB compaction process lazily applies that intent to the original value. This feature is described in detail in [Merge Operator](#)

🔗 DB ID

A globally unique ID created at the time of database creation and stored in `IDENTITY` file in the `DB` folder by default. Optionally it can only be stored in the `MANIFEST` file. Storing in the `MANIFEST` file is recommended.

🔗 5. Tools

There are a number of interesting tools that are used to support a database in production. The `sst_dump` utility dumps all the keys-values in a `sst` file, as well as other information. The `ldb` tool can put, get, scan the contents of a database. `ldb` can also dump contents of the `MANIFEST`, it can also be used to change the number of configured levels of the database. See [Administration and Data Access Tool](#) for details.

🔗 6. Tests

There are a bunch of unit tests that test specific features of the database. A `make check` command runs all unit tests. The unit tests trigger specific features of RocksDB and are not designed to test data correctness at scale. The `db_stress` test is used to validate data correctness at scale. See [Stress-test](#). You can also add fuzzers to [this directory](#) by leveraging the [fuzzing](#) infrastructure there, the fuzzers are continuously run on Google's [OSS-Fuzz](#), errors will be reported to RocksDB developers.

🔗 7. Performance

RocksDB performance is benchmarked via a utility called `db_bench`. `db_bench` is part of the RocksDB source code. Performance results of a few typical workloads using Flash storage are described [here](#). You can also find RocksDB performance results for in-memory workload [here](#).

🔗 Author: Dhruva Borthakur et al.

Contents

- [RocksDB Wiki](#)
- [Overview](#)
- [RocksDB FAQ](#)
- [Terminology](#)
- [Requirements](#)
- [Contributors' Guide](#)
- [Release Methodology](#)
- [RocksDB Users and Use Cases](#)
- [RocksDB Public Communication and Information Channels](#)
- [Basic Operations](#)
 - [Iterator](#)
 - [Prefix seek](#)
 - [SeekForPrev](#)
 - [Tailing Iterator](#)
 - [Compaction Filter](#)
 - [Multi Column Family Iterator](#)
 - [Read-Modify-Write \(Merge\) Operator](#)
 - [Column Families](#)
 - [Creating and Ingesting SST files](#)
 - [Single Delete](#)
 - [SST Partitioner](#)
 - [Low Priority Write](#)
 - [Time to Live \(TTL\) Support](#)
 - [Transactions](#)
 - [Snapshot](#)
 - [DeleteRange](#)
 - [Atomic flush](#)
 - [Read-only and Secondary instances](#)
 - [Approximate Size](#)
 - [User-defined Timestamp](#)
 - [Wide Columns](#)
 - [BlobDB](#)
 - [Online Verification](#)
- **Options**
 - [Setup Options and Basic Tuning](#)
 - [Option String and Option Map](#)
 - [RocksDB Options File](#)
- [MemTable](#)
- [Journal](#)
 - [Write Ahead Log \(WAL\)](#)
 - [Write Ahead Log File Format](#)
 - [WAL Recovery Modes](#)
 - [WAL Performance](#)
 - [WAL Compression](#)
 - [MANIFEST](#)

- [Track WAL in MANIFEST](#)
- **Cache**
 - [Block Cache](#)
 - [SecondaryCache \(Experimental\)](#)
- [Write Buffer Manager](#)
- [Compaction](#)
 - [Leveled Compaction](#)
 - [Universal compaction style](#)
 - [FIFO compaction style](#)
 - [Manual Compaction](#)
 - [Subcompaction](#)
 - [Choose Level Compaction Files](#)
 - [Managing Disk Space Utilization](#)
 - [Trivial Move Compaction](#)
 - [Remote Compaction](#)
- **SST File Formats**
 - [Block-based Table Format](#)
 - [PlainTable Format](#)
 - [CuckooTable Format](#)
 - [External Table](#)
 - [Index Block Format](#)
 - [Bloom Filter](#)
 - [Data Block Hash Index](#)
- [IO](#)
 - [Rate Limiter](#)
 - [SST File Manager](#)
 - [Direct I/O](#)
- [Compression](#)
 - [Dictionary Compression](#)
- [Full File Checksum and Checksum Handoff](#)
- [Background Error Handling](#)
- [Huge Page TLB Support](#)
- [Tiered Storage \(Experimental\)](#)
- **Logging and Monitoring**
 - [Logger](#)
 - [Statistics](#)
 - [Compaction Stats and DB Status](#)
 - [Perf Context and IO Stats Context](#)
 - [EventListener](#)
- [Known Issues](#)
- [Troubleshooting Guide](#)
- [Tests](#)
 - [Stress Test](#)
 - [Fuzzing](#)
 - [Benchmarking](#)
- **Tools / Utilities**
 - [Administration and Data Access Tool](#)
 - [How to Backup RocksDB?](#)
 - [Replication Helpers](#)
 - [Checkpoints](#)
 - [How to persist in-memory RocksDB database](#)

- [Third-party language bindings](#)
- [RocksDB Trace, Replay, Analyzer, and Workload Generation](#)
- [Block cache analysis and simulation tools](#)
- [IO Tracer and Parser](#)
- **Implementation Details**
 - [Delete Stale Files](#)
 - [Partitioned Index/Filters](#)
 - [WritePrepared-Transactions](#)
 - [WriteUnprepared-Transactions](#)
 - [How we keep track of live SST files](#)
 - [How we index SST](#)
 - [Merge Operator Implementation](#)
 - [RocksDB Repairer](#)
 - [Write Batch With Index](#)
 - [Two Phase Commit](#)
 - [Iterator's Implementation](#)
 - [Simulation Cache](#)
 - [\[To Be Deprecated\] Persistent Read Cache](#)
 - [DeleteRange Implementation](#)
 - [unordered_write](#)
- **Extending RocksDB**
 - [RocksDB Configurable Objects](#)
 - [The Customizable Class](#)
 - [Object Registry](#)
- **RocksJava**
 - [RocksJava Basics](#)
 - [Logging in RocksJava](#)
 - [JNI Debugging](#)
 - [RocksJava API TODO](#)
 - [RocksJava Performance on Flash Storage](#)
 - [Tuning RocksDB from Java](#)
- **Performance**
 - [Performance Benchmarks](#)
 - [In Memory Workload Performance](#)
 - [Read-Modify-Write \(Merge\) Performance](#)
 - [Delete A Range Of Keys](#)
 - [Write Stalls](#)
 - [Pipelined Write](#)
 - [MultiGet Performance](#)
 - [Tuning Guide](#)
 - [Memory usage in RocksDB](#)
 - [Speed-Up DB Open](#)
 - [Implement Queue Service Using RocksDB](#)
 - [Asynchronous IO](#)
 - [Off-peak in RocksDB](#)
- [Projects Being Developed](#)
- **Misc**
 - [Building on Windows](#)
 - [Developing with an IDE](#)
 - [Open Projects](#)

- [Talks](#)
- [Publication](#)
- [Features Not in LevelDB](#)
- [How to ask a performance-related question?](#)
- [Articles about Rocks](#)

Clone this wiki locally

`https://github.com/facebook/rocksdb.wiki.git`

